МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

«Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ

по лабораторной работе №3

по дисциплине

«Информационные и технические средства защиты авторских прав»

«Электронная цифровая подпись RSA»

Выполнил:
Лебедевич Артем Владимирович
магистрант группы 556301

Преподаватель:
Ярмолик Вячеслав Николаевич
доктор технических наук
профессор кафедры ПОИТ

Минск, 2026 г.

СОДЕРЖАНИЕ

1 Цель работы 1

2 Теоретические сведения 2
2.1 Функция хеширования 3
2.2 Классическая схема цифровой подписи 3
2.3 Алгоритм цифровой подписи RSA 4
2.4 Пример из методического пособия 4

3 Реализация 5
3.1 Структура программы 6
3.2 Ключевые фрагменты кода 6
3.3 Интерфейс командной строки 7
3.4 Графическое приложение 7

4 Результаты работы программы 7

5 Анализ стойкости 9
5.1 Коллизии хеш-функции при малом модуле 10
5.2 Слабости учебной хеш-функции 10
5.3 Мультипликативность RSA 10
5.4 Цифровая подпись в защите авторских прав 11

Заключение 12

Список использованных источников 13

1 ЦЕЛЬ РАБОТЫ

Изучить назначение и устройство функций хеширования и электронной цифровой подписи. В соответствии с вариантом №1 (нечётный номер в списке группы) реализовать программу вычисления и проверки электронной цифровой подписи по схеме RSA, используя для вычисления хеш-образа сообщения функцию (3.2) методического пособия. Проверить корректность реализации на примере из пособия и исследовать стойкость схемы.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Функция хеширования

Функцией хеширования h называется преобразование, переводящее строку M произвольной длины в значение $m = h(M)$ фиксированной длины — хеш-образ, или дайджест сообщения. Криптографическая хеш-функция должна:

- быть чувствительной к любому изменению входной последовательности;
- применяться к данным любой длины и давать результат фиксированной длины;
- быть односторонней: по $h(M)$ невозможно найти M;
- не допускать нахождения второго прообраза $M' \neq M$ с $h(M') = h(M)$;
- не допускать нахождения произвольной пары $M_1 \neq M_2$ с $h(M_1) = h(M_2)$ (коллизии).

Хеш-образ строится поблочно: $H_i = f(H_{i-1}, m_i)$, результат обработки последнего блока становится хеш-образом всего сообщения. В пособии в качестве учебного примера используется квадратичная функция

$$H_i = (H_{i-1} + m_i)^2 \bmod n, \tag{2.1}$$

где $n = p \cdot q$, H_0 — произвольное начальное значение, m_i — код i-го символа: А $= 1$, Б $= 2, \ldots,$ Я $= 33$.

Для сообщения $M =$ «БГУИР» $= \{2, 4, 21, 10, 18\}$ при $n = 17 \cdot 19 = 323$ и $H_0 = 100$ вычисления приведены в таблице 2.1.

Таблица 2.1 – Хеш-образ сообщения «БГУИР»

i	Символ	m_i	$H_i = (H_{i-1} + m_i)^2 \bmod 323$
1	Б	2	$(100 + 2)^2 \bmod 323 = 68$
2	Г	4	$(68 + 4)^2 \bmod 323 = 16$
3	У	21	$(16 + 21)^2 \bmod 323 = 77$
4	И	10	$(77 + 10)^2 \bmod 323 = 140$
5	Р	18	$(140 + 18)^2 \bmod 323 = 93$

Таким образом, $h($БГУИР$) = 93$.

2.2 Классическая схема цифровой подписи

Цифровая подпись играет для электронного документа ту же роль, что подпись от руки для бумажного: подтверждает, что документ составил или заверил владелец подписи, и позволяет обнаружить изменение документа

после подписания. Схема решает три задачи: подпись невозможно подделать, её принадлежность субъекту можно проверить, а сам субъект не может от неё отказаться.

Отправитель, владеющий секретным ключом K_c:

1. вычисляет хеш-образ $m = h(M)$;
2. вычисляет подпись S по хеш-образу с помощью секретного ключа;
3. отправляет пару (M, S).

Получатель пары (M', S), знающий открытый ключ K_o:

1. вычисляет хеш-образ полученного сообщения $m' = h(M')$;
2. извлекает из подписи хеш-образ m с помощью открытого ключа;
3. признаёт подпись подлинной, если $m' = m$.

Подписывается не само сообщение, а его хеш-образ: это делает подпись короткой независимо от длины документа и избавляет от необходимости выполнять медленное возведение в степень для каждого блока сообщения.

2.3 Алгоритм цифровой подписи RSA

Пара ключей вычисляется так же, как для криптосистемы RSA (лабораторная работа №2): $r = p \cdot q$, $\varphi(r) = (p-1)(q-1)$, открытая экспонента e взаимно проста с $\varphi(r)$, секретная d удовлетворяет условию $e \cdot d \bmod \varphi(r) = 1$. Открытый ключ $K_o = (e, r)$ передаётся партнёрам для проверки подписей, секретный $K_c = (d, r)$ хранит автор.

Роли ключей по сравнению с шифрованием меняются местами: подпись вычисляется секретным ключом, а проверяется открытым:

$$S = m^d \bmod r, \qquad m = S^e \bmod r. \tag{2.2}$$

Подпись подлинна, если $h(M') = S^e \bmod r$. Корректность следует из того же тождества $m^{ed} \equiv m \pmod r$, что и для шифрования. Создать подпись, которая пройдёт проверку открытым ключом, может только владелец d, а вычислить d по (e, r) так же трудно, как разложить r на множители.

2.4 Пример из методического пособия

Для $p = 17$, $q = 19$: $r = 323$, $\varphi(r) = 288$. При $e = 43$ расширенный алгоритм Евклида даёт $d = 67$ (таблица 2.2). Открытый ключ $(43, 323)$, секретный — $(67, 323)$.

Таблица 2.2 – Расширенный алгоритм Евклида для $a = 288$, $b = 43$

Шаг	q	d	x	y
0	—	288	1	0
1	—	43	0	1
2	6	30	1	-6
3	1	13	-1	7
4	2	4	3	-20
5	3	1	-10	67

Подпись сообщения «БГУИР»: $S = 93^{67} \bmod 323 = 206$, отправляется пара $\{\text{БГУИР}, 206\}$. Пусть при передаче сообщение изменено и получатель принял $\{\text{БРУИР}, 206\}$. Он вычисляет $h(\text{БРУИР}) = 123$, а из подписи восстанавливает $206^{43} \bmod 323 = 93$. Значения не совпадают, подпись признаётся недействительной.

В пособии хеш-функция и подпись используют один и тот же модуль $n = r = 323$. В программе это сделано правилом: хеш вычисляется по модулю r, поэтому любой хеш-образ является допустимым блоком RSA ($m < r$) при любых ключах.

3 РЕАЛИЗАЦИЯ

3.1 Структура программы

Код третьей лабораторной работы находится в отдельном пакете `lab3`. Ключи RSA вычисляются модулем `lab2/rsa.py`, а быстрое возведение в степень и расширенный алгоритм Евклида — общими функциями `helpers/numtheory.py` (листинг 3.1).

Листинг 3.1 – Структура программы

```
main.py                CLI: команды sign и verify, demo 3, gui 3
configs/cfg.py         p, q, e, H0 по умолчанию (пример из пособия)
helpers/numtheory.py   fast_exp, euclid_ex (общие с ЛР2)
lab2/rsa.py            вычисление пары ключей RSA
lab3/hashing.py        квадратичная хеш-функция (формула 3.2)
lab3/signature.py      вычисление и проверка подписи
lab3/runner.py         прогон примеров пособия, коллизия, самопроверка
lab3/gui.py            окно: подписание, проверка, пошаговый разбор
```

3.2 Ключевые фрагменты кода

Хеш-функция (листинг 3.2) кодирует символы по таблице пособия А $=$ $1, \ldots,$ Я $=$ 33 (с буквой Ё). Чтобы подписывать произвольные документы, таблица продолжена латинскими буквами ($A = 34, \ldots, Z = 59$) и цифрами ($0 = 60, \ldots, 9 = 69$). Регистр не учитывается, пробелы и знаки препинания не имеют кода и пропускаются.

Листинг 3.2 – Квадратичная хеш-функция

```
RUSSIAN = "АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
ENGLISH = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
DIGITS = "0123456789"
CODES = {symbol: i for i, symbol in enumerate(RUSSIAN + ENGLISH + DIGITS, start=1)}


def quad_hash(text, n, h0, trace=None):
    h = h0 % n
    for symbol in (ch for ch in text.upper() if ch in CODES):
        m = CODES[symbol]
        h = (h + m) ** 2 % n
        if trace is not None:
            trace.append(HashStep(symbol, m, h))
    return h
```

Вычисление и проверка подписи (листинг 3.3) занимают несколько строк: вся сложность сосредоточена в функциях `fast_exp` и `make_keys`, реализованных и проверенных в лабораторной работе №2. Проверка возвращает оба сравниваемых значения, чтобы программа могла показать, в чём именно расхождение.

Листинг 3.3 – Вычисление и проверка подписи

```
class Verdict(NamedTuple):
    computed: int         # m' = h(M') from the received message
    recovered: int        # m = S^e mod r from the signature
    valid: bool


def sign(text, d, r, h0, trace=None):
    return fast_exp(quad_hash(text, r, h0, trace), d, r)


def verify(text, signature, e, r, h0, trace=None):
    if not 0 <= signature < r:
        raise ValueError(f"The signature must be in 0..r-1 = 0..{r - 1}")
    computed = quad_hash(text, r, h0, trace)
    recovered = fast_exp(signature, e, r)
    return Verdict(computed, recovered, computed == recovered)
```

3.3 Интерфейс командной строки

Команды `sign` и `verify` (листинг 3.4) принимают ключи либо через p, q, e, либо напрямую: автору для подписи достаточно (d, r), получателю для проверки — открытого ключа (e, r). Флаг `-v` выводит вычисление хеш-образа по шагам. При недействительной подписи команда `verify` завершается с кодом 1, что позволяет использовать её в скриптах.

Листинг 3.4 – Примеры запуска

```
python3 main.py sign -t БГУИР -v               # хеш по шагам, h(M) = 93, S = 206
python3 main.py sign -d 67 -r 323 -t БГУИР     # то же по секретному ключу
python3 main.py verify -e 43 -r 323 -s 206 -t БГУИР    # VALID
python3 main.py verify -e 43 -r 323 -s 206 -t БРУИР    # INVALID: 123 != 93
python3 main.py demo 3                          # все примеры с проверкой
python3 main.py gui 3                           # графическое окно
```

3.4 Графическое приложение

Окно лабораторной работы №3 (`python3 main.py gui 3`) построено на базовом классе окна лабораторной работы №2 и дополнительно содержит поле H_0, поле сообщения, поле подписи и кнопки «Подписать» и «Проверить». При подписании вычисленная подпись подставляется в поле S, на панели разбора выводятся ход алгоритма Евклида, таблица хеширования и формула $S = m^d \bmod r$ с числами. При проверке результат выводится цветом: зелёным для подлинной подписи и красным для недействительной, с обоими сравниваемыми значениями. Это позволяет на занятии изменить одну букву сообщения или одну цифру подписи и сразу увидеть, как изменился хеш-образ.

4 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

Команда `python3 main.py demo 3` выполняет пример из пособия, поиск коллизии, подпись длинного сообщения 256-битным ключом и самопроверку. Сводные результаты приведены в таблице 4.1, полный вывод — в листинге 4.1.

Таблица 4.1 – Результаты работы программы

Проверка	Результат	Эталон
Секретная экспонента, $p = 17$, $q = 19$, $e = 43$	$d = 67$	совпадает
Хеш-образ «БГУИР»	93	совпадает
Подпись «БГУИР»	$S = 206$	совпадает
Проверка $\{\text{БГУИР}, 206\}$	подлинная	—
Проверка $\{\text{БРУИР}, 206\}$	$123 \neq 93$, отклонена	совпадает
Проверка $\{\text{БГУИР}, 207\}$	$93 \neq 24$, отклонена	—
Коллизия $\{\text{АБЛ}, 206\}$	принята как подлинная	—
256-битный ключ, изменена одна цифра	отклонена	—
200 случайных сообщений и ключей	все подлинные	—
Изменение одного символа в них	отклонено в 200 из 200	—

Листинг 4.1 – Вывод программы

```
=== Lab 3: RSA digital signature ===

=== Example from the lab manual ===

p = 17, q = 19, r = 323, phi(r) = 288, e = 43
Extended Euclid:
step      q        d        x        y
   0             288        1        0
   1              43        0        1
   2      6       30        1       -6
   3      1       13       -1        7
   4      2        4        3      -20
   5      3        1      -10       67
d = 67; matches the lab manual
Ko = (43, 323), Kc = (67, 323)

--- Hash of БГУИР (n = r = 323) ---
H0 = 100
H1 = (H0 + m1)^2 mod n = (100 + 2)^2 mod 323 = 68    [Б]
H2 = (H1 + m2)^2 mod n = (68 + 4)^2 mod 323 = 16    [Г]
H3 = (H2 + m3)^2 mod n = (16 + 21)^2 mod 323 = 77    [У]
H4 = (H3 + m4)^2 mod n = (77 + 10)^2 mod 323 = 140    [И]
H5 = (H4 + m5)^2 mod n = (140 + 18)^2 mod 323 = 93    [Р]
h(M) = 93; matches the lab manual

S = m^d mod r = 93^67 mod 323 = 206; matches the lab manual
Signed message: {БГУИР, 206}

--- Genuine message ---
```

```
M' = БГУИР
S = 206
H0 = 100
H1 = (H0 + m1)^2 mod n = (100 + 2)^2 mod 323 = 68    [Б]
H2 = (H1 + m2)^2 mod n = (68 + 4)^2 mod 323 = 16    [Г]
H3 = (H2 + m3)^2 mod n = (16 + 21)^2 mod 323 = 77    [У]
H4 = (H3 + m4)^2 mod n = (77 + 10)^2 mod 323 = 140    [И]
H5 = (H4 + m5)^2 mod n = (140 + 18)^2 mod 323 = 93    [Р]
m' = h(M') = 93
m = S^e mod r = 93
signature is VALID

--- Message changed in transit ---
M' = БРУИР
S = 206
H0 = 100
H1 = (H0 + m1)^2 mod n = (100 + 2)^2 mod 323 = 68    [Б]
H2 = (H1 + m2)^2 mod n = (68 + 18)^2 mod 323 = 290    [Р]
H3 = (H2 + m3)^2 mod n = (290 + 21)^2 mod 323 = 144    [У]
H4 = (H3 + m4)^2 mod n = (144 + 10)^2 mod 323 = 137    [И]
H5 = (H4 + m5)^2 mod n = (137 + 18)^2 mod 323 = 123    [Р]
m' = h(M') = 123
m = S^e mod r = 93
signature is INVALID; matches the lab manual

--- Signature changed in transit ---
M' = БГУИР
S = 207
H0 = 100
H1 = (H0 + m1)^2 mod n = (100 + 2)^2 mod 323 = 68    [Б]
H2 = (H1 + m2)^2 mod n = (68 + 4)^2 mod 323 = 16    [Г]
H3 = (H2 + m3)^2 mod n = (16 + 21)^2 mod 323 = 77    [У]
H4 = (H3 + m4)^2 mod n = (77 + 10)^2 mod 323 = 140    [И]
H5 = (H4 + m5)^2 mod n = (140 + 18)^2 mod 323 = 93    [Р]
m' = h(M') = 93
m = S^e mod r = 24
signature is INVALID

=== Weakness of a small modulus: hash collision ===

h(АБЛ) = h(БГУИР) = 93, found after 46 tries
--- Forged message with the stolen signature ---
M' = АБЛ
S = 206
H0 = 100
H1 = (H0 + m1)^2 mod n = (100 + 1)^2 mod 323 = 188    [А]
H2 = (H1 + m2)^2 mod n = (188 + 2)^2 mod 323 = 247    [Б]
H3 = (H2 + m3)^2 mod n = (247 + 13)^2 mod 323 = 93    [Л]
m' = h(M') = 93
m = S^e mod r = 93
signature is VALID

=== 256-bit modulus, e = 65537 ===

r = 82924805423338389897214052819514817915331336743115935277657371471556577865171
d = 74709129368381096069260212317857590505495779209675118509057976282995080571873

--- Genuine message ---
M' = Лебедевич Артем Владимирович, лабораторная работа 3, 2026
S = 29096725476915481606490077833955416584088312737526206206732455365896115536116
m' = h(M') = 73967938765295933998387524035571872905223564277412855834493943216929750470872
m = S^e mod r = 73967938765295933998387524035571872905223564277412855834493943216929750470872
signature is VALID

--- One digit changed ---
M' = Лебедевич Артем Владимирович, лабораторная работа 4, 2026
S = 29096725476915481606490077833955416584088312737526206206732455365896115536116
m' = h(M') = 53354868048707820284393711651565680685151016449043082107106414119054689609572
m = S^e mod r = 73967938765295933998387524035571872905223564277412855834493943216929750470872
signature is INVALID

Self-check: 200 random messages and keys, every signature verified;
a one-symbol change detected in 200 of 200 cases
```

5 АНАЛИЗ СТОЙКОСТИ

5.1 Коллизии хеш-функции при малом модуле

Стойкость подписи не выше стойкости хеш-функции: подпись связана с сообщением только через хеш-образ, поэтому любое другое сообщение с тем же хеш-образом примет чужую подпись как свою. При $n = 323$ хеш-функция принимает не более 323 значений, и коллизию можно найти простым перебором. Программа перебирает трёхбуквенные слова и уже на 46-й попытке находит слово «АБЛ» с $h(\text{АБЛ}) = 93 = h(\text{БГУИР})$. Пара $\{\text{АБЛ}, 206\}$ проходит проверку как подлинная, хотя владелец ключа никогда не подписывал это сообщение.

По парадоксу дней рождения для хеш-образа длиной k бит коллизия находится примерно за $2^{k/2}$ попыток, поэтому в реальных системах используются хеш-функции длиной не менее 256 бит (SHA-256, SHA-3, белорусский стандарт СТБ 34.101.31 «belt-hash»).

5.2 Слабости учебной хеш-функции

Квадратичная хеш-функция пособия не является криптографической и при большом модуле. Каждый её шаг — возведение в квадрат, а у квадрата по модулю $n = pq$ четыре корня: значения x и $n - x$ дают одинаковый результат, поэтому два разных промежуточных значения $H_{i-1} + m_i$, дающие в сумме n, приводят к одному хеш-образу. Зная разложение n, можно извлекать квадратные корни и обращать шаги хеш-функции. Если хеш вычисляется по модулю r ключа подписи, этим знанием обладает сам автор: он может заранее построить пару сообщений с одинаковым хеш-образом, подписать одно и затем утверждать, что подписывал другое. Кроме того, в реализации пробелы и знаки препинания не участвуют в хешировании, поэтому сообщения «БГУ ИР» и «БГУИР» имеют одинаковую подпись. Таблица кодов пособия определена только для букв, и это ограничение сохранено сознательно.

5.3 Мультипликативность RSA

Без хеширования подпись RSA допускает подделку: из $S_1 = m_1^d$ и $S_2 = m_2^d$ любой получает подпись $S_1 S_2 \bmod r = (m_1 m_2)^d \bmod r$ сообщения $m_1 m_2$, не зная секретного ключа, а для любого S число $S^e \bmod r$ является «сообщением», которому S подходит как подпись. Хеширование разрушает эту алгебраическую связь, поскольку противник должен ещё найти осмысленный документ с нужным хеш-образом. Современный стандарт RSA-PSS дополнительно добавляет к хеш-образу случайную соль.

5.4 Цифровая подпись в защите авторских прав

Для защиты авторских прав цифровая подпись решает две задачи: подтверждает авторство (подпись под произведением может поставить только владелец секретного ключа) и целостность (любое изменение произведения после подписания обнаруживается). В сочетании с меткой времени от доверенной службы подписанный хеш-образ произведения служит доказательством того, что автор обладал им на определённую дату. Подпись, однако, не мешает копированию: её можно удалить вместе с файлом-приложением, поэтому для контроля распространения она дополняется встраиваемыми водяными знаками и средствами управления цифровыми правами.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены требования к функциям хеширования и классическая схема электронной цифровой подписи. Реализована программа вычисления и проверки подписи по схеме RSA (вариант №1) с хеш-функцией (3.2) пособия, с интерфейсом командной строки и графическим окном, показывающим вычисления по шагам. Корректность подтверждена совпадением всех значений с примером пособия ($d = 67$, $h(M) = 93$, $S = 206$, $h(\text{БРУИР}) = 123$) и проверкой на 200 случайных сообщениях и ключах.

Эксперимент с коллизией показал, что стойкость подписи определяется слабейшим звеном схемы: при 323 возможных значениях хеш-образа подделка находится за 46 попыток, и стойкость RSA при этом не играет роли. Надёжная подпись требует одновременно модуля RSA не менее 2048 бит, криптографической хеш-функции длиной не менее 256 бит и схемы дополнения хеш-образа, например RSA-PSS.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ярмолик, С. В. Криптография и охрана коммерческой информации: метод. пособие по выполнению лаб. работ / С. В. Ярмолик, В. Н. Ярмолик. — Минск: БГУИР, 2010. — 32 с.
2. Rivest, R. A method for obtaining digital signatures and public-key cryptosystems / R. Rivest, A. Shamir, L. Adleman // Communications of the ACM. — 1978. — Vol. 21, No. 2. — P. 120–126.
3. Шнайер, Б. Прикладная криптография. Протоколы, алгоритмы, исходные тексты на языке Си / Б. Шнайер. — М.: Триумф, 2002. — 816 с.
4. Столлингс, В. Криптография и защита сетей: принципы и практика / В. Столлингс. — М.: Вильямс, 2001. — 672 с.